

TP2 – Projeto de Bloco

Dashboard de Sensores ARM→FPGA

1. Introdução

Este segundo Teste de Performance dá continuidade ao desenvolvimento do projeto Dashboard de Sensores ARM→FPGA iniciado no TP1. O objetivo do projeto é construir um sistema embarcado capaz de adquirir informações de sensores ambientais através do Raspberry Pi Zero 2W, transmitir os dados para a FPGA Tang Nano 4K e apresentar os resultados em dispositivos de visualização como LEDs e display OLED.

Durante o TP2 foram desenvolvidos os primeiros módulos digitais relacionados ao projeto, além da evolução da arquitetura ARM–FPGA e da implementação de rotinas em Assembly ARM64 para simulação do processamento inicial dos dados dos sensores.

O trabalho também contemplou simulações em Verilog, síntese de hardware na FPGA e depuração de software utilizando GDB.

2. Objetivos do TP2

Os principais objetivos desta etapa foram:

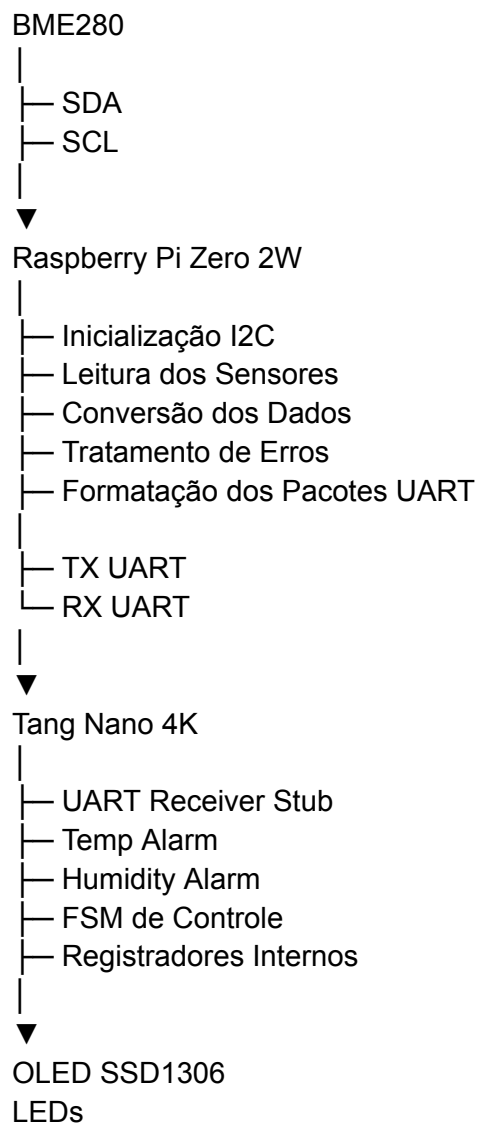
- Refinar a arquitetura ARM–FPGA do sistema.
 - Desenvolver módulos digitais em Verilog relacionados ao monitoramento de sensores.
 - Validar os módulos através de testbenches e simulações.
 - Realizar síntese dos módulos utilizando a ferramenta Gowin IDE.
 - Desenvolver rotinas em Assembly ARM64.
 - Demonstrar o uso de ferramentas de depuração.
 - Definir uma estratégia inicial de comunicação entre Raspberry Pi e FPGA.
-

3. Arquitetura Atualizada do Sistema

3.1 Arquitetura Geral

BME280
↓ I2C
Raspberry Pi Zero 2W
↓ UART
Tang Nano 4K
↓
OLED SSD1306 / LEDs

3.2 Arquitetura Detalhada



3.3 Divisão de Responsabilidades

Raspberry Pi Zero 2W

Responsável por:

- Leitura do sensor BME280.
- Processamento inicial dos dados.
- Conversão dos valores físicos.
- Controle geral do sistema.
- Transmissão dos dados para a FPGA.

FPGA Tang Nano 4K

Responsável por:

- Recepção dos dados via UART.
 - Processamento digital dedicado.
 - Geração de alarmes.
 - Controle dos LEDs.
 - Controle do display OLED.
 - Execução de FSMs.
-

4. Desenvolvimento em Verilog

4.1 Módulo Temp Alarm

O módulo Temp Alarm foi desenvolvido para identificar temperaturas acima de um limite pré-definido.

Código

```
module temp_alarm(  
  
    input [7:0] temperatura,  
  
    output alarm  
  
);  
  
    assign alarm = (temperatura > 8'd30);  
  
endmodule
```

Funcionamento

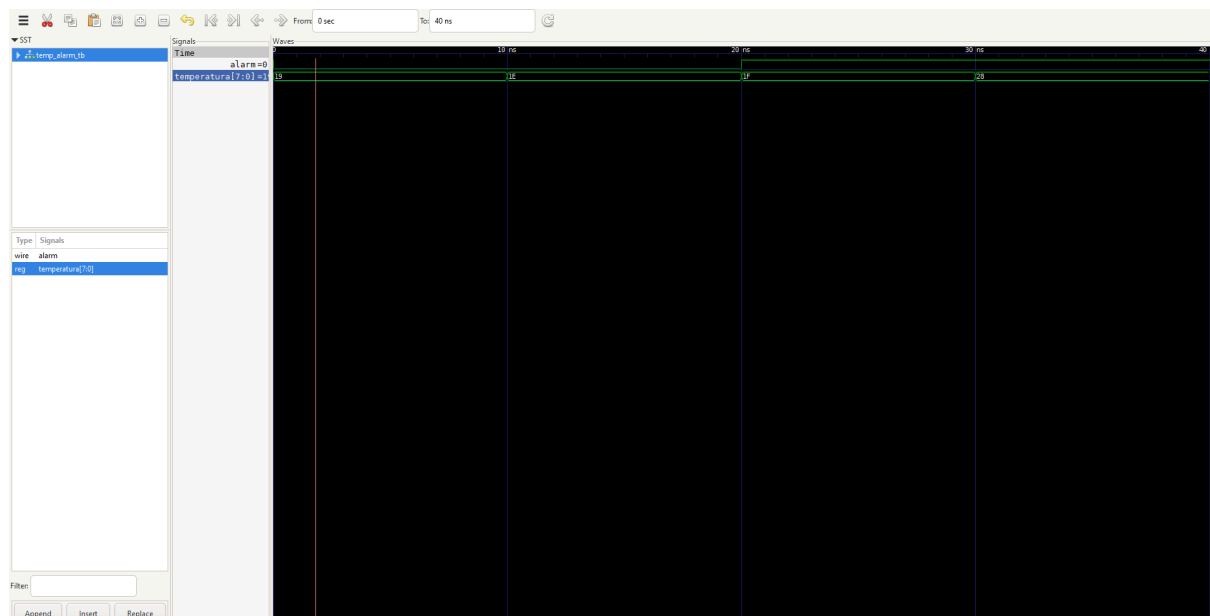
Quando a temperatura ultrapassa 30°C o sinal de saída alarm assume nível lógico alto.

Casos de Teste

Temperatura	Alarm
25°C	0
30°C	0
31°C	1
40°C	1

Resultado

A simulação demonstrou o correto acionamento do alarme para temperaturas superiores ao limite estabelecido.



4.2 Módulo Humidity Alarm

O módulo Humidity Alarm foi desenvolvido para monitorar níveis elevados de umidade.

Código

```
module humidity_alarm(  
  
    input [7:0] humidity,  
  
    output alarm
```

```
);
```

```
assign alarm = (humidity > 8'd70);
```

```
endmodule
```

Funcionamento

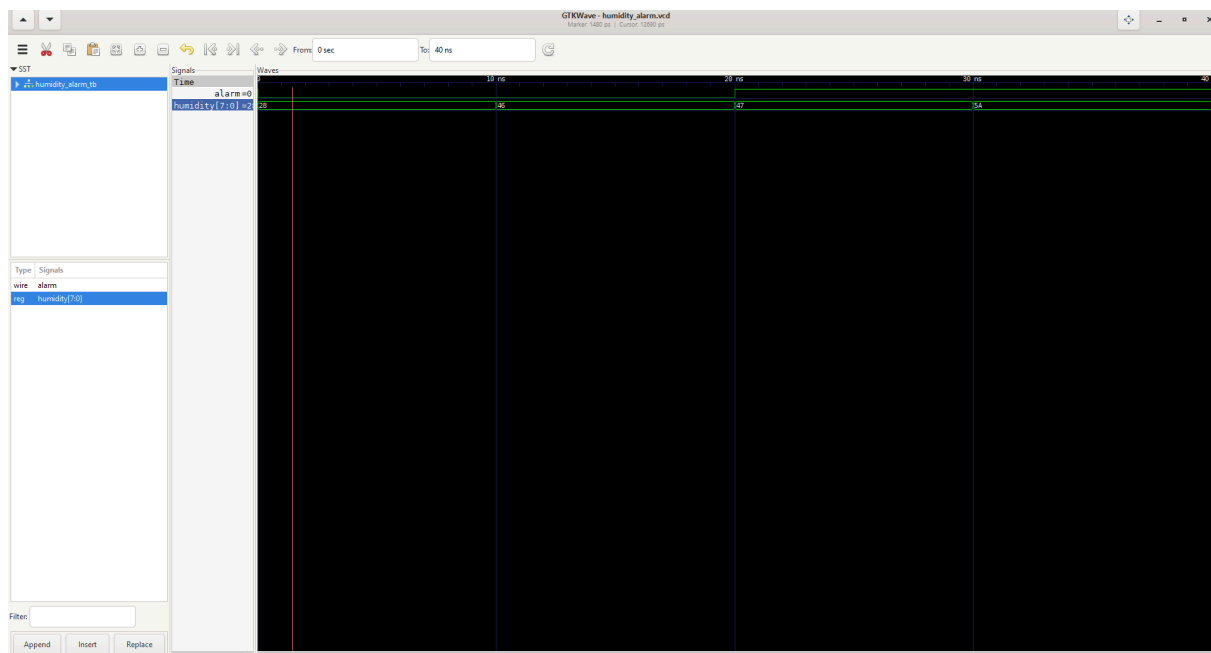
O alarme é acionado quando a umidade ultrapassa 70%.

Casos de Teste

Umidade	Alarm
40%	0
70%	0
71%	1
90%	1

Resultado

O comportamento observado na simulação correspondeu ao esperado para todos os casos testados.



4.3 Módulo UART Receiver Stub

Com o objetivo de preparar a futura comunicação entre Raspberry Pi e FPGA foi desenvolvido um receptor UART simplificado.

Código

```
module uart_receiver_stub(  
  
    input rx,  
  
    output reg data_ready  
  
);  
  
always @(*) begin  
  
    if(rx == 1'b0)  
        data_ready = 1'b1;  
    else  
        data_ready = 1'b0;  
  
end  
  
endmodule
```

Objetivo

Este módulo representa uma estrutura inicial para recepção dos pacotes UART que serão enviados pelo Raspberry Pi nas próximas etapas do projeto.

5. Simulações e Validação

As simulações foram realizadas utilizando Icarus Verilog e GTKWave.

Foram desenvolvidos testbenches específicos para cada módulo.

Os resultados obtidos demonstraram:

- Funcionamento correto dos comparadores de temperatura.
 - Funcionamento correto dos comparadores de umidade.
 - Geração adequada dos sinais de saída.
 - Validação dos requisitos funcionais previstos.
-

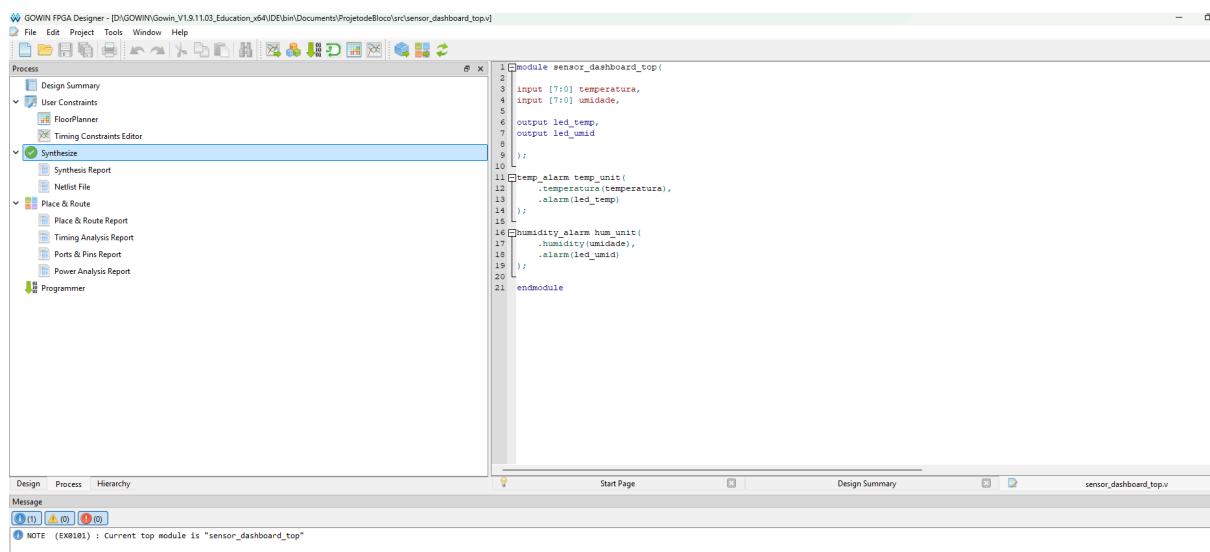
6. Síntese na FPGA Tang Nano 4K

Os módulos foram sintetizados utilizando a ferramenta Gowin IDE.

A síntese ocorreu sem erros ou warnings, demonstrando compatibilidade com a FPGA Tang Nano 4K.

Resultados observados:

- Síntese concluída com sucesso.
- Top Module identificado corretamente.
- Estrutura compatível com implementação física.



7. Desenvolvimento em Assembly ARM64

7.1 Programa Sensor Monitor

Foi desenvolvido um programa Assembly ARM64 para simular a leitura de sensores ambientais.

O programa utiliza:

- Registradores ARM64.
- Instruções LDR.
- Instruções STR.
- Comparações CMP.
- Desvios condicionais.

- Manipulação de memória.

Objetivo

Simular a leitura dos seguintes valores:

- Temperatura: 35°C
- Umidade: 75%

Comparando-os com limites definidos previamente.

Resultado Esperado

- Temp Alert = 1
- Umid Alert = 1

Conceitos Demonstrados

- Manipulação de memória.
 - Controle de fluxo.
 - Comparação de dados.
 - Geração de flags de alerta.
-

8. Depuração com GDB

A depuração foi realizada utilizando GDB Multiarch em conjunto com QEMU ARM64.

Foram executadas as seguintes operações:

- Conexão remota ao QEMU.
- Execução passo a passo com a instrução SI.
- Inspeção de registradores.
- Verificação da execução do programa.

Ferramentas utilizadas:

- QEMU ARM64
 - GDB Multiarch
 - GNU Assembler
 - GNU Linker
-

9. Comunicação ARM–FPGA

Foi definida uma proposta inicial de protocolo UART para troca de informações entre o Raspberry Pi e a FPGA.

Exemplo de Pacotes

TEMP:35

HUM:75

Fluxo de Comunicação

BME280

↓

Raspberry Pi

↓

UART

↓

UART Receiver Stub

↓

Temp Alarm / Humidity Alarm

↓

OLED e LEDs

Esta estrutura servirá como base para a implementação da comunicação real nos próximos TPs.

10. Conclusão

Durante o TP2 foi possível evoluir significativamente o projeto Dashboard de Sensores ARM-FPGA.

Foram desenvolvidos e validados módulos digitais relacionados ao monitoramento ambiental, criada uma proposta inicial de comunicação entre ARM e FPGA e implementadas rotinas Assembly ARM64 para simulação do processamento dos dados dos sensores.

Os resultados obtidos demonstram que a arquitetura proposta permanece viável e preparada para as próximas etapas do projeto.

11. Planejamento para o TP3

No TP3 serão desenvolvidas as seguintes atividades:

- Implementação de FSM de controle.
- Evolução do receptor UART.
- Integração inicial ARM ↔ FPGA.
- Leitura real do sensor BME280.
- Atualização dos LEDs com dados reais.
- Primeira exibição em display OLED.
- Testes de comunicação em hardware real.

Essas atividades representarão a primeira integração funcional entre os componentes principais do sistema.